https://doi.org/10.1007/s11704-025-41136-3

RESEARCH ARTICLE

SEA-SQL: semantic-enhanced text-to-SQL with adaptive refinement

Chaofan LI[1], Yingxia SHAO[1]✉, Yawen LI[2], Zheng LIU[3]

1. School of Computer Science (National Pilot Software Engineering School), Beijing University of Posts and Telecommunications, Beijing 100876, China
2. School of Economics and Management, Beijing University of Posts and Telecommunications, Beijing 100876, China
3. Beijing Academy of Artificial Intelligence, Beijing 100190, China

Received October 22, 2024; accepted January 9, 2025

E-mail: shaoyx@bupt.edu.cn

© Higher Education Press 2026

Abstract

Recent advancements in large language models (LLMs) have significantly contributed to the progress of the Text-to-SQL task. A common requirement in many of these works is the post-correction of SQL queries. However, the majority of this process entails analyzing error cases to develop prompts with rules that eliminate model bias. And there is a weakness of execution verification for SQL queries. In addition, the prevalent techniques primarily depend on GPT-4 and few-shot prompts, resulting in expensive costs. To investigate the effective methods for SQL refinement in a cost-efficient manner, we introduce Semantic-Enhanced Text-to-SQL with Adaptive Refinement (SEA-SQL), which includes Adaptive Bias Elimination and Dynamic Execution Adjustment, aims to improve performance while minimizing resource expenditure with zero-shot prompts. Specifically, SEA-SQL employs a semantic-enhanced schema to augment database information and optimize SQL queries. During the SQL query generation, a fine-tuned adaptive bias eliminator is applied to mitigate inherent biases caused by the LLM. The dynamic execution adjustment is utilized to guarantee the executability of the bias eliminated SQL query. We conduct experiments on the Spider and BIRD datasets to demonstrate the effectiveness of this framework. The results demonstrate that SEA-SQL achieves state-of-the-art performance in the GPT-3.5 scenario with 9%–58% of the generation cost. Furthermore, SEA-SQL is comparable to GPT-4 with only 0.9%–5.3% of the generation cost. Our code is available at the website of github.com/545999961/SEA-SQL.

Keywords

Text-to-SQL; adaptive bias elimination; dynamic execution adjustment; economize

1 Introduction

Relational databases play a pivotal role in information technology by offering a structured and organized framework for data storage. Through tables, rows, and columns, data is efficiently organized, ensuring clarity and ease of management. However, users typically interact with databases through SQL queries, which is a barrier for those unfamiliar with SQL syntax. To alleviate the gap between natural language and SQL queries, the Text-to-SQL task is proposed [1]. Specifically, given a natural language question, Text-to-SQL aims to convert the question into a SQL query, making it more convenient for users to interact with database.

Traditional sequence-to-sequence methods mainly focus on fine-tuned encoder-decoder models [2–4], which are optimized for the encoding process and decoding process, including schema linking [5,6], building intermediate results between the natural language and SQL [7–9], decoding constraints [10–12], etc. As databases continue to expand in size, they often encompass a large number of tables and columns, complex inter-table relationships, diverse data types, and vast amounts of data. Consequently, user queries tend to exhibit complex structures, such as nested queries, aggregations, joins, or multiple conditions [13]. Both factors increase the complexity of inputs for the Text-to-SQL task. Sequence-to-sequence models have inherent limitations due to the small size of their parameters. As the complexity of input increases, these models may struggle to encode all necessary information efficiently, leading to performance bottlenecks.

Recently, with the rise of LLMs, there has been a significant advancement in natural language understanding, contextual awareness, and scalability. LLMs are capable of effectively managing and interpreting complex queries and schema structures. This has made prompt-based Text-to-SQL approaches more accessible and effective. By simply providing prompts, LLMs can effortlessly get SQL queries and obtain outstanding performance [14–19]. However, previous works in employing LLMs for Text-to-

SQL prompting have three major limitations. (1) **Inherent model bias**: When generating SQL queries, models may exhibit biases due to patterns present in their training data. Such biases might manifest in specific preferences within the generated queries. For example, GPT-3.5 is case-insensitive and may inadvertently convert an uppercase value from the input into a lowercase value in the SQL query [14]. This behavior reflects the model's inherent biases and can lead to inaccuracies in query formation. (2) **Unexecutable SQL**: Although SQL is generated by the LLM, they lack the capability to verify the executability of the final SQL query. When executed, these queries may encounter various errors, such as column mismatches or runtime errors. This limitation underscores the need for additional validation steps to ensure the generated SQL is functional and accurate. REACT [20] demonstrates that the LLM may generate illusory answers in question answering (QA) scenarios without external retrieval capabilities, incorporating retrieval into LLM generation enhances the accuracy and reliability of answer generation. Similarly, the LLM in the Text-to-SQL domain can also generate non-executable SQL queries. Therefore, the absence of external actions such as SQL execution limits the assurance of the executability of the generated SQL. (3) **Expensive inference cost**: Recent works [17,18,21] focus on leveraging GPT-4 for SQL generation. However, utilizing GPT-4 comes at a significant expense. Furthermore, most GPT-4 based methods [17,18,21] rely on few-shot prompts, which leads to longer inputs and higher computational costs. For instance, DAIL-SQL [18] requires nearly fanxiexian_myfh0.1 per SQL query, MAC-SQL [21] requires fanxiexian_myfh0.2 per SQL query, and DIN-SQL [17] requires nearly fanxiexian_myfh0.8 to generate each SQL query. If these methods are employed to generate thousands or even tens of thousands of SQL statements, it would be a catastrophic cost for ordinary users.

To mitigate these issues, we introduce Semantic-Enhanced Text-to-SQL with Adaptive Refinement (SEA-SQL), an innovative framework based on zero-shot prompts and GPT-3.5 to accomplish the Text-to-SQL task in an economical manner. Our framework incorporates a semantic-enhanced schema to enrich database information and refine SQL queries through adaptive bias elimination and dynamic execution adjustment. Specifically, we augment the schema with question-related column values to enhance its semantic understanding. We then fine-tune and employ a bias eliminator to mitigate the inherent biases of the LLM during SQL generation. With dynamic execution adjustment, it ensures the executability of SQL queries by interleaving reflection and correction within the execution process.

We evaluate the effectiveness of our approach by conducting evaluations on multiple datasets, including the Spider dataset and its variant dataset, as well as the BIRD dataset. Our framework demonstrates remarkable performance on the Spider dataset, achieving an accuracy of 83.6% on the development set. This surpasses the state-of-the-art (SOTA) GPT-3.5-based method and is competitive to the performance of GPT-4. In addition, when testing on the Spider variant dataset, specifically Spider-Realistic development set, our framework exhibits outstanding performance. Furthermore, on the BIRD dataset, our framework achieves an execution accuracy of 56.13% on the development set, outperforming all GPT-3.5-based methods with only 9%−58% generation cost. This result is on par with the performance of other GPT-4-based methods with only 0.9%−5.3% generation cost, highlighting the competitive capabilities of our approach. The main contributions of our work are as follows:

- We propose an economically efficient and highly effective framework SEA-SQL for leveraging LLMs to accomplish Text-to-SQL Task with zero-shot prompts.
- We propose adaptive bias elimination and dynamic execution adjustment to mitigate potential errors in SQL queries and improve the performance of the Text-to-SQL task.
- SEA-SQL significantly outperforms previous SOTA GPT-3.5 based Text-to-SQL method, and obtains an excellent performance competitive to GPT-4 with lower cost.

2 Related work

With the rise of LLMs, individuals are progressively exploring the applications of LLMs, achieving successful outcomes particularly in the Text-to-SQL domain. For example, ChatGPT's Evaluation [14] conducts a comprehensive evaluation about zero-shot performance of GPT-3.5's ability in Text-to-SQL task. C3 [16] proposes a zero-shot method utilizing GPT-3.5 to generate SQL, improving the generated SQL by providing clear layouts, calibrating bias prompts, and enhancing it through self-consistency [22]. DAIL-SQL [18] investigates various prompt representations and few-shot example selection methods to determine the most effective approach for efficient SQL generation. DIN-SQL [17] divides SQL generation into three stages: schema linking, SQL difficulty decomposition, and post-processing to refine SQL. MAC-SQL [21] transforms the task of Text-to-SQL into a multi-agent collaboration task, constructs a selector for schema linking, a decomposer to break down questions and generate SQL, and a refiner to ensure executable SQL.

However, there are some issues with the methods mentioned earlier. Firstly, their ability to eliminate model bias is restricted to the model itself. This is achieved either by constraining model generation via prompts [16], or by creating rule-based prompts to correct SQL [17,21]. Since the model may not always adhere to prompts or recognize its own deficiencies, this strategy has limitations in eliminating model bias. Secondly, while MAC-SQL proposes ensuring SQL executability through execution, it identifies only five types of errors that the model may encounter in the prompt based on specific error cases. Consequently, the model's effectiveness in addressing unknown errors is restricted, thereby limiting the model's overall capability. Thirdly, most methods utilize GPT-4 for SQL generation, which comes with expensive costs. Although C3 [16] exclusively utilizes GPT-3.5, it suffers from the drawback of inferior performance.

Additionly, DAIL-SQL [18] presents a method to fine-tune open-source LLMs for the Text-to-SQL task. However, their performance is notably inferior, displaying a significant gap compared to other models in the GPT series.

3 Task definition

Given a question Q_{qes} and a database schema S consisting of:

1. A set of tables $T = t_1, t_2, \ldots, t_N$, where N is the total number of tables. Each t_i represents the ith table.
2. A set of columns $C = c_1^1, \ldots, c_{n_1}^1, \ldots, c_1^N, \ldots, c_{n_N}^N$, where n_i denotes the number of columns in table t_i, and c_j^i represents the jth column of table t_i.
3. A set of values $V = v_1^1, \ldots, v_m^1, \ldots, v_1^M, \ldots, v_m^M$, where $M = \sum_{i=1}^{N} n_i$ is the total number of columns. Each column has m sample values, and v_k^j denotes the kth value of column c^j.
4. A set of foreign keys $R = \{(t_i.c_k = t_j.c_l)\}$, where $t_i, t_j \in T, c_k, c_l \in C$.

The Text-to-SQL task is to generate an SQL A based on the question Q_{qes} and the schema S. This generated SQL can be executed on the database to get the answer of the question Q_{qes}.

4 Methodology

To optimize the Text-to-SQL task by LLMs and ensure the generation of executable SQL that effectively produces the correct results, we propose SEA-SQL. As illustrated in Fig. 1, our framework incorporates a semantic-enhanced schema to enrich the database information and refine SQL queries through adaptive bias elimination and dynamic execution adjustment with zero-shot prompts.

Specifically, the semantic-enhanced schema includes column values related to the question, which facilitates the LLM's identification and utilization of the appropriate tables and columns during SQL query generation. Subsequently, an adaptive bias eliminator, which is a fine-tuned small LLM (e.g., Mistral-7B), is used to eliminate the inherent biases of the LLM within the SQL queries, thereby enhancing the quality of the generated SQL queries. Finally, with dynamic execution adjustment, an iterative process of SQL execution and LLM reflection-correction is employed until the SQL query is executed successfully.

4.1 Semantic-enhanced schema

The study [23] points out the limitations of solely relying on table and column names, along with their foreign key relations, to grasp the semantics of a database schema with respect to a specific question. It is demonstrated that such an approach potentially overlooks crucial contextual information contained within the actual data values of the database. Therefore, it becomes necessary to integrate the actual values present in the database into the SQL generation process to mitigate this limitation. While previous research [23] suggests augmenting the schema with values extracted from the columns mentioned in the question, this approach encounters challenges, particularly when the question does not explicitly specify the necessary values. For instance, consider a question like "the average salary of America", where the "country" column only contains the value "USA". In such cases, without supplemental information, it is difficult to accurately deduce the intended value format of the question.

To enable the LLM to understand the database comprehensively and utilize table names, column names, and value information correctly, we propose a simple yet effective method [24]. Although the "bridging" method [23] yields excellent performance, its application is limited by slow processing speeds when handling large datasets. Additionally, it extracts only the values explicitly mentioned in the question, which can result in insufficient information capture. To address these limitations, we propose a new retrieval paradigm that combines BM25 [25] with the "bridging" method [26]. In this integrated approach, "bridging" functions as a second-stage retrieval mechanism, enhancing the overall efficiency and depth of information retrieval.

Initially, we construct a BM25 index for each column of the database, leveraging the inherent structure and relationships within the data. Using this index, we retrieve a subset of the top-k sample values (e.g., k=1000) from each column in the database. We then apply the "bridging" method [23] for string matching. If any of the top-k values perfectly match specific words in a given question, these values are moved to the new top-1 position. Finally, we select the top-m values (e.g., m=2) from these rearranged lists as sampled values. We apply these sampled values to enhance the semantic representation of the database schema. As shown in Fig. 2, compared to a standard schema, semantic-enhanced schema retrieves the top-m values of columns relevant to the question, assisting the LLM in better understanding the database content. This ensures that even in cases of ambiguous column names or particularly unique format column values, the LLM can still comprehend the database information accurately, enabling correct usage of table names, column names, and values.

The LLM then generates an SQL query based on both the original question Q_{qes} and the database schema enhanced with semantic

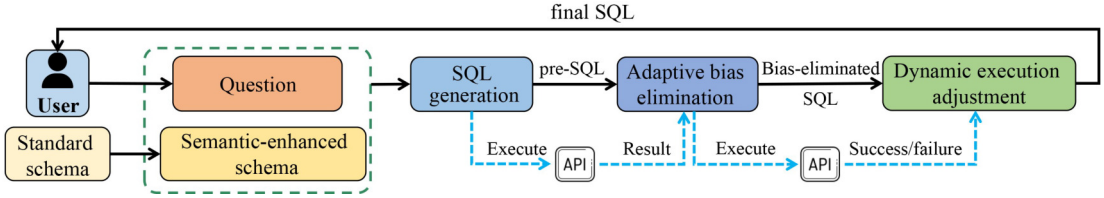


Fig. 1 An overview of the SEA-SQL framework: The SQL query is generated using the semantic-enhanced schema. Then, through adaptive bias elimination, the inherent biases caused by the LLM are eliminated. Finally, with dynamic execution adjustment, the SQL query is ensured to execute successfully to return the expected results to the user

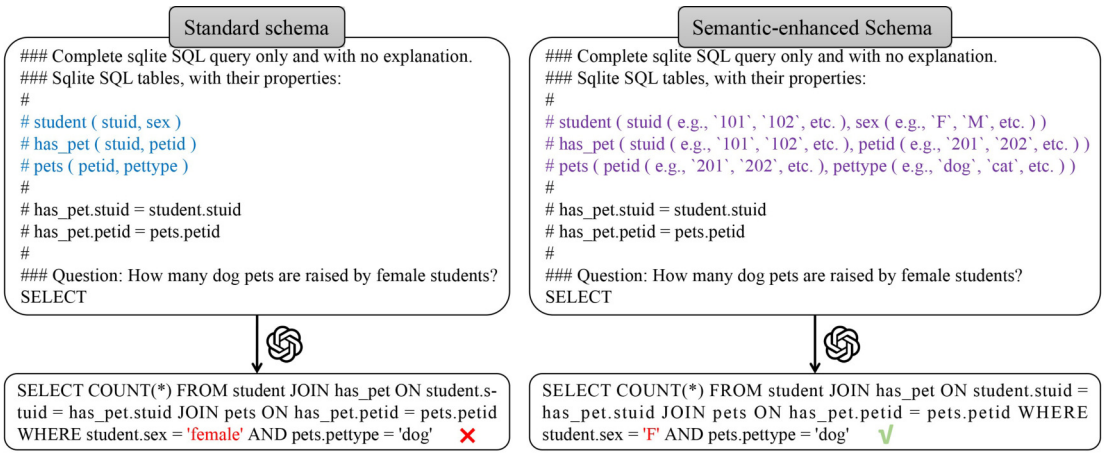


Fig. 2 Comparison of different schema formats. The left shows a standard schema, while the right showcases a semantic-enhanced schema

information S, following the zero-shot prompt format proposed by [14], as detailed in Appendix C:

$$A_{pre} = \text{LLM}_{\text{generate}}\left(Q_{qes}, S\right), \tag{1}$$

where A_{pre} represents the preliminary SQL query (pre-SQL) generated by the LLM with semantic-enhanced schema. This pre-SQL serves as the foundation for the subsequent steps in SEA-SQL.

4.2 Adaptive bias elimination

Due to inherent biases, LLMs may generate SQL queries that contain specific model-related errors. These errors often occur because the models tend to favor certain patterns influenced by biases present in the training data, e.g., GPT-3.5 prefer to use "LEFT JOIN" rather than "JOIN" [14,16]. While approaches like DIN-SQL [17] suggest using specific prompts to address biases, they are based on few-shot examples and manually add rules in prompts from the error case analysis, which may not cover all potential error cases and may not correct the errors effectively. To comprehensively eliminate the errors caused by the LLM inherent biases, we propose an adaptive bias eliminator, which adaptively discovers and corrects the inherent biases caused by the SQL generation.

- **Objective** Previous study [18] indicates that a fine-tuned small LLM (e.g., Mistral-7B) does not yield comparable performance to GPT-3.5 or GPT-4 for directly generating SQL queries. The challenges encountered by the small LLM is attributed to the complexities involved in understanding SQL syntax and database information. In this paper, we propose a simplified task for the small LLM, suggesting it act as a bias eliminator of potential biases that might be introduced by the large SQL generation model (e.g., GPT-3.5). Therefore the small LLM does not need to fully comprehend all aspects of SQL syntax or complex database information. The primary role transition to identifying and eliminating inherent biases in the SQL generation model, ensures the production of high-quality and unbiased SQL queries.

Specifically, the bias eliminator receives two types of information: (1) SQL queries generated by the SQL generation model (pre-SQL), and (2) the execution results of queries include the execution information (success or failure) and their corresponding results. The former provides the coarse-grained information for the bias eliminator, while the latter offers a fine-grained information of the generated SQL queries, helping in the detection of potential biases in the generated SQL queries. The bias eliminator then outputs a revised SQL query, which removes any identified biases:

$$A_{elim} = M_{elim}\left(Q_{qes}, S, A_{pre}, r\right), \tag{2}$$

where A_{elim} denotes the bias-eliminated SQL, M_{elim} represents the adaptive bias eliminator, and r stands for the corresponding execution result of A_{pre}. If an error occurs during execution and no results are obtained, the content of r will be the error message.

- **Fine-tuning** As shown in Fig. 3, to support the adaptive bias eliminator in identifying and mitigating biases introduced by the SQL generation LLM effectively, it must be exposed to a substantial number of bias cases produced by the SQL generation LLM. To obtain such training data, we conduct an extension to the standard training pairs: $(Q^t_{qes}, S^t, A^t_{gold})$, where t indicates the training data, Q^t_{qes} represents the question in the train set and A^t_{gold} denotes the correct SQL query corresponding to the question Q^t_{qes} and S^t represents the semantic-enhanced schema in the train set. Concretely, we use the SQL generation LLM to generate the pre-SQL A^t_{pre} for each question, and obtain the execution result r^t for each A^t_{pre}. The

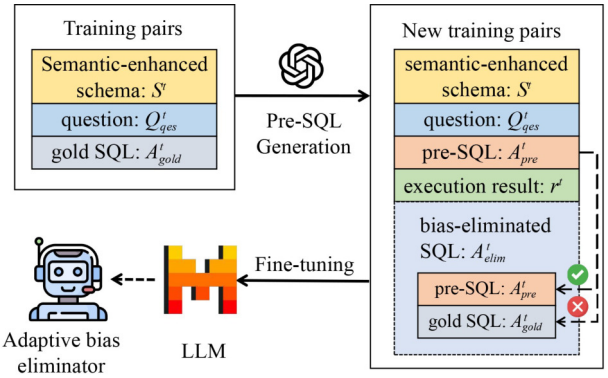


Fig. 3 The fine-tuning process of Adaptive Bias Elimination. A checkmark (√) indicates that the execution result of A^t_{pre} is identical to the execution result of A^t_{gold}, while a crossmark (×) indicates they differ

supplementary data r^t provides the bias eliminator with a comprehensive understanding of where the SQL generation model is prone to making errors because of the inherent model biases.

We expect the adaptive bias eliminator to be capable of modifying SQL queries that contain model bias, while preserving the original format of queries that are free from such bias. If a pre-SQL query yields execution results identical to those of a gold SQL query, then the bias eliminator should leave the pre-SQL query unchanged, denoted as $A^t_{elim} = A^t_{pre}$. Conversely, if the pre-SQL query execution results deviate from those of the gold SQL query, the bias eliminator should adjust the query to match the gold, reflected by $A^t_{elim} = A^t_{gold}$. This ensures that modifications are only made when necessary and that the integrity of bias-free queries is maintained. Finally, the new training pairs are expressed as: $(Q^t_{qes}, S^t, A^t_{pre}, r^t, A^t_{elim})$.

The training objective for the small LLM bias eliminator, denoted as M_{elim} is to maximize the likelihood of output A^t_{elim} across all T training pairs:

$$\mathcal{L} = -\sum^{T} \log p_{M_{elim}}(A^t_{elim} | Q^t_{qes}, S^t, A^t_{pre}, r^t). \quad (3)$$

4.3 Dynamic execution adjustment

Although SQL queries may be free of biases, they can still fail to execute due to issues such as column mismatches or runtime errors. These issues are not directly related to bias, but they determine whether the SQL can ultimately execute successfully, which is a crucial aspect of the Text-to-SQL task. However, a significant portion of prior research concentrates on the refinement of SQL exclusively through the LLM, ignoring whether the execution is successful or not [17]. As depicted in Fig. 4, we propose dynamic execution adjustment, which corrects unexecutable SQL queries by leveraging the LLM to both identify the error reason in the SQL query and regenerate it for execution. Inspired by Chain-of-Thought (CoT) [27] and REACT [20], we divide the adjustment process into three steps: execute, reflect, and correct. Through external execution of SQL API calls and continuous internal reflection and correction within the LLM, we ensure that the SQL query eventually can be successfully executed.

Execute In this initial step, the SQL query is executed through external API calls to the database. This execution provides real-world feedback on the effectiveness of the SQL query. If the execution is successful, no further action is needed. However, if the execution fails, it indicates that there's an issue with the SQL query that requires correction.

Reflect Once encountering a failed execution, the LLM reflects on the error encountered. This involves analyzing the error message from a failed execution and identifying the specific reason of the SQL query that caused the failure. This reflective process facilitates the LLM in understanding how the SQL query needs to be adjusted or corrected.

Correct Based on the insights gained from reflection, the LLM regenerates the SQL query, thereby addressing the identified errors. This correction process involves making appropriate adjustments to the SQL syntax, structure, or logic to correct the underlying problem. The goal is to produce a revised SQL query that resolves the issues identified during reflection.

Detailed zero-shot prompts of these steps can be found in Appendix C. Figure 5 illustrates an example of dynamic execution adjustment process, the LLM dynamically corrects errors in SQL queries by utilizing error feedback and its reflection information. When an error is detected by execution, the LLM conducts a thorough reflection of its error reason and try to correct it, leading to the generation of a refined SQL query. This refined query is then undergoes another execution attempt. If additional errors emerge during this iterative process, the LLM methodically reflects upon and corrects the SQL iteratively until a version capable of successful execution is obtained.

By following the three-step approach of execute, reflect, and correct, it iteratively improves the quality and effectiveness of the generated SQL queries. Continuous feedback from the execution results and dynamic adjustments guided by the LLM ensure that the SQL queries evolve to be more accurate and executable over time. Note that, throughout the iterative process, the LLM memories previous error information, reflections, and correction results to avoid it regenerating previously erroneous SQL queries during dynamic execution adjustments.



Fig. 4 The process of dynamic execution adjustment

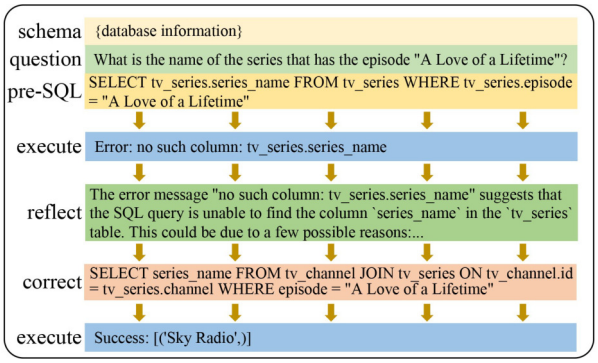


Fig. 5 An example of dynamic execution adjustment

5 Experiment

5.1 Experiment setup

Datasets The Spider dataset, as described in [28], is a comprehensive semantic parsing and Text-to-SQL dataset of significant scale and complexity. This dataset comprises 10,181 questions and 5,693 unique complex SQL queries. It covers 200 datasets featuring multiple tables and spans 138 diverse domains. It comprises 8,659 instances for the training set and 1,034 instances for the development set.

The BIRD dataset, introduced in [13], serves as a substantial benchmark for large-scale database tasks related to Text-to-SQL. This dataset includes 12,751 pairs of Text-to-SQL data and encompasses 95 databases, with a collective size of 33.4 GB. Covering 37 professional domains, BIRD stands out as the first Text-to-SQL benchmark to emphasize efficiency, encouraging the development of more efficient query methods tailored for massive and noisy database contents. It comprises 9,428 instances for the training set, 1,534 instances for the development set, and 1,789 instances for the concealed test set.

To ensure the robustness of our model, we conduct a thorough evaluation using another distinct dataset: Spider-Realistic [29] consisting of 508 samples. This dataset is curated from the Spider dataset, wherein questions are strategically modified to emulate real-world application scenarios. Specifically, Spider-Realistic omits explicitly mentioned column names in questions. It comprises 508 instances for the development set.

Evaluation metrics To evaluate the performance of the Text-to-SQL task, following [30] and [13], we adopt two metrics: Execution Accuracy (EX) and Valid Efficiency Score (VES).

- The Execution Accuracy (EX) is the proportion of examples in the evaluation dataset where the execution results of the predicted SQL queries are the same as those of the gold SQL queries. For an execution result Y_n of a gold SQL denoted by V_n, and an execution result of a predicted SQL $\hat{Y}_n$ denoted by $\hat{V}_n$, EX is computed using the formula:

$$\mathrm{EX} = \frac{\sum_{n=1}^{N} \Phi(V_n, \hat{V}_n)}{N},$$

where N represents the total number of SQL queries, and the indicator function $\Phi(\cdot)$ is defined as:

$$\Phi(V, \hat{V}) = \begin{cases} 1, & \text{if } V = \hat{V}, \\ 0, & \text{if } V \neq \hat{V}. \end{cases}$$

- The Valid Efficiency Score (VES) could assess the efficiency of valid SQL queries generated by models. A "valid SQL" is defined as a predicted SQL query whose execution result matches its corresponding gold SQL result. Queries that fail to generate correct execution results are deemed invalid as they are ineffective for user needs, irrespective of efficiency. Therefore, the VES metric takes into account both the effectiveness and the efficiency of execution, providing a comprehensive measure of model performance. The VES can be calculated as follows:

$$\mathrm{VES} = \frac{\sum_{n=1}^{N} \Phi(V_n, \hat{V}_n) \cdot R(Y_n, \hat{Y}_n)}{N},$$

where $R(\cdot)$ evaluates the relative execution efficiency of the predicted SQL in relation to the gold SQL:

$$R(Y_n, \hat{Y}_n) = \sqrt{\frac{E(Y_n)}{E(\hat{Y}_n)}}.$$

here $E(\cdot)$ is a function that assesses the execution efficiency of each SQL in a specific setting [13].

LLMs We employ two distinct LLMs for our task: GPT-3.5, a black-box model accessible solely through the API, and Mistral-7B, an open-source model that allows us not only to utilize it via the API but also affords the flexibility to freely fine-tune its parameters. We employ GPT-3.5[1)] for SQL query generation and dynamic execution adjustment. To mitigate biases in GPT-3.5, we utilize a fine-tuned Mistral-7B model as our adaptive bias eliminator.

Implementation details The Mistral-7B model is fine-tuned using LoRA [31], deepspeed [32], and flash attention [33], employing a batch size of 64 and a learning rate of 2e-4. The training is conducted on the Spider and BIRD training datasets for one epoch using 8 × 32G V100 GPUs in 1.5 hours. Additionally, we incorporate a linear warm-up with 10% of the training steps and employ linear decay to adjust the learning rate during training. For semantic-enhanced schema, we retrieve the top 1000 sample values and retain the top 2 after "bridging". To prevent the system from entering an infinite loop, we have capped the maximum iterations for dynamic execution adjustment at 10.

Baselines We conducted experiments on both the BIRD and Spider datasets, comparing our method with the following baselines:

- **GPT-4 [17]** utilizes a straightforward zero-shot Text-to-SQL prompt for SQL generation.
- **C3 [16]** employs clear prompting, calibration with hints, and ensures consistent output through self-consistency to generate SQL queries.
- **DIN-SQL [17]** decomposes the generation task into smaller sub-problems. The solutions to these sub-problems are then inputted into LLMs to produce the final SQL output.
- **DAIL-SQL [18]** leverages few-shot prompting by retrieving examples from the training set to facilitate the generation of SQL queries.
- **MAC-SQL [21]** completes the text-to-SQL task through a multi-agent collaboration approach, involving schema selection, question decomposition, and SQL refinement.
- **T5-3B [11]** fine-tunes the T5-3B model for text-to-SQL tasks.
- **T5-3B + PICARD [11]** fine-tunes the T5-3B model for text-to-SQL tasks, with PICARD applying syntax rules to ensure

[1)] We use gpt-3.5-turbo-16k-0613 to conduct our experiment.

the generated SQL is parsed incrementally, verifying each step against predefined syntax and structural constraints.

- **Supervised Fine-Tuning [18]** involves directly fine-tuning LLMs specifically for the text-to-SQL task.

5.2 Overall performance

Spider result Table 1 presents the performance comparison between our proposed framework and baseline approaches on the Spider dataset's development set. Notably, it's essential to mention that Spider has closed its test set for evaluation. Leveraging GPT-3.5, SEA-SQL outperforms other methods that either use GPT-3.5 or are fine-tuned, achieving a high EX score on the development set.

A distinguishing factor of our approach is the use of zero-shot prompts, as opposed to the few-shot prompts that is predominantly used in other methods. This highlights the simplicity and effectiveness of our approach.

Compared to fine-tuned approaches [18], the utilization of a small LLM as a bias eliminator is more effective than directly employing a small LLM to generate SQL. In comparison to GPT-3.5 based approaches, SEA-SQL significantly outperforms them, emphasizing the effectiveness of combining GPT-3.5 with fine-tuning as a promising direction for further exploration. Moreover, the results remain competitive with DIN-SQL and DAIL-SQL in the scenario of GPT-4, demonstrating SOTA performance based on GPT-3.5 and maintaining competitiveness relative to GPT-4.

BIRD result Table 2 showcases the performance of our framework on the BIRD dataset. SEA-SQL demonstrates substantial performance superiority over other GPT-3.5-based methods. Particularly noteworthy is its competitiveness with DIN-SQL and DAIL-SQL on the development set. Our approach surpasses the

Table 1 SEA-SQL and baselines on Spider development set (EX result), where the best outcomes are highlighted in bold, while the second highest results are underlined

Method	Model	Zero-shot	Few-shot	Fine-tune	dev
GPT-4 [17]	GPT-4	√			72.9
DIN-SQL [17]	GPT-4		√		82.8
DAIL-SQL [18]	GPT-4		√		83.5
MAC-SQL [21]	GPT-4		√		**86.8**
C3 [16]	GPT-3.5	√			81.8
DIN-SQL [17]	GPT-3.5		√		74.9
DAIL-SQL [18]	GPT-3.5		√		78.2
MAC-SQL [21]	GPT-3.5		√		80.6
T5-3B [11]	T5-3B			√	74.4
T5-3B + PICARD [11]	T5-3B			√	79.9
Supervised Fine-Tuning [18]	Llama2-7b			√	66.7
Supervised Fine-Tuning [18]	Llama2-13b			√	67.0
SEA-SQL(ours)	GPT-3.5 + Mistral-7B	√		√	<u>83.6</u>

Table 2 SEA-SQL and baselines on BIRD

Method	Model	Zero-shot	Few-shot	Fine-tune	dev		Test	
					EX	VES	EX	VES
GPT-4	GPT-4		√		46.35	49.77	54.89	60.77
DIN-SQL	GPT-4		√		50.72	58.79	55.90	59.44
DAIL-SQL	GPT-4		√		54.76	56.08	<u>57.41</u>	<u>61.95</u>
MAC-SQL	GPT-4		√		**57.56**	58.76	**59.59**	**67.68**
DIN-SQL	GPT-3.5		√		44.92	46.73	–	–
DAIL-SQL	GPT-3.5		√		44.65	46.99	–	–
MAC-SQL	GPT-3.5		√		50.56	**61.25**	–	–
SEA-SQL(ours)	GPT-3.5 + Mistral-7B	√		√	<u>56.13</u>	<u>60.83</u>	53.66	57.00

current best GPT-3.5-based MAC-SQL EX result by a significant margin of 11.02% on the development set, thereby establishing a new SOTA performance based on GPT-3.5.

5.3 Ablation study

We conducted a series of ablations to investigate the pivotal components of our framework. Four sets of ablation experiments were performed on the BIRD development set, each targeting a specific aspect of our approach: the adaptive bias elimination (ABE), dynamic execution adjustment (DEA), and both the ABE and DEA.

Table 3 presents the experimental results of the ablation study. Additionally, it illustrates the model's performance across varying complexities of SQL problems, categorized as Simple, Moderate (Mod.), and Challenging (Chall.). These categories are defined by human experts and consider factors such as Question Understanding, Knowledge Reasoning, Data Complexity, and SQL Complexity [13].

The ablation study on the BIRD dataset underscores the importance of both the adaptive bias elimination and the dynamic execution adjustment components in handling SQL queries of varying complexity. Removing either component significantly reduced performance, and eliminating both drastically worsened the outcomes. The impact of removing these components was particularly severe for moderate and complex SQL queries, while simpler SQLs experienced a smaller decline in performance. This suggests that our proposed SEA-SQL framework is particularly beneficial for generating more complex SQL queries. In the case of the Spider dataset, the DEA component did not influence the final results due to the dataset's relative simplicity. All generated bias-eliminated SQL queries were executable, rendering the dynamic execution adjustment superfluous in improving performance.

Overall, the results definitely underscore the crucial role played by each component. Notably, the absence of any module consistently resulted in a noticeable decrease in overall performance. This highlights the interdependence of the modules within our framework and underscores their collective significance in attaining optimal results.

5.4 Results on different sample values

To investigate the impact of the semantic-enhanced schema on performance, we conducted comparative analyses between the standard schema and the semantic-enhanced schema, utilizing varying numbers of sample values. Given that modifying the number of samples necessitates retraining of the adaptive bias eliminator, our evaluation focused solely on the performance of pre-SQL. Due to the excessive number of sample values, the input may become too lengthy, making it challenging for the LLM to accurately comprehend the database information. Consequently, we conducted tests using 0, 1, 2, and 3 samples, as illustrated in the Table 4. The results provided significant insights into the importance of sample values in the effectiveness of our methodology.

When the number of values is zero, essentially using standard schema, a notable 17% decrease is observed. This reduction underscores the significance of our semantic-enhanced schema in achieving optimal results.

While using only two sample values yielded the best results exclusively for the moderate complexity level of the BIRD dataset, this approach closely approached optimal performance for both the simple and complex complexity levels. Additionally, the average performance achieved by using two sample values surpassed that obtained with either one or three sample values across both the BIRD

Table 3 The results of ablation studies on the development set

Method	BIRD								Spider
	EX				VES				
	Simple	Mod.	Chall.	All	Simple	Mod.	Chall.	All	EX
SEA-SQL	**64.11**	**47.74**	**31.94**	**56.13**	**69.63**	**51.71**	**33.72**	**60.83**	**83.6**
w/o ABE	62.49	45.81	30.56	54.43	66.45	49.98	31.32	58.16	81.8
w/o DEA	62.27	43.44	27.28	53.32	67.80	46.72	29.64	57.83	**83.6**
w/o ABE & DEA	59.57	37.85	21.53	49.41	63.98	41.03	22.50	53.13	80.9

Table 4 The results of the number of different sample values on the development set

Values num	BIRD								Spider
	EX				VES				
	Simple	Mod.	Chall.	All	Simple	Mod.	Chall.	All	EX
0	49.51	30.11	21.53	41.00	55.65	30.20	22.13	44.79	76.4
1	59.24	36.34	**22.22**	48.83	63.55	38.47	**22.53**	52.10	79.5
2	**59.57**	**37.85**	21.53	**49.41**	**63.98**	**41.03**	22.50	**53.13**	**80.9**
3	59.14	36.99	20.83	48.83	61.97	40.88	23.29	51.95	78.5

and Spider datasets. This improvement likely occurs because two values offer more informative content than a single value, while three values can make the input overly long, potentially degrading performance.

5.5 Cost analysis

One primary concern in the development of Text-to-SQL technology is its accessibility to ordinary users [34–36]. To address this concern, we conducted an evaluation of our approach, comparing it against established baselines in terms of cost effectiveness (up to May 2024)[2], as outlined in Table 5, where "Pre" denotes the cost of pre-processing, "Generate" refers to the cost required to generate each SQL query, and "All" represents the total cost when generating all SQL queries. "GPU-Time" denotes the duration each query utilizes GPU, "API-Time" refers to the API request time for each SQL query, and "Full-Time" indicates the total time required to generate a complete SQL query. Our findings reveal a significant cost advantage with our framework over the GPT-4 based approach. Since SEQ-SQL only uses 25 GB of GPU and takes 1.09 seconds to generate a single SQL, this generation overhead can be considered negligible. Although our GPT-3.5 model incurs slightly higher costs with other GPT-3.5 based methods, primarily attributed to pre-processing, the expense is significantly mitigated after training the bias eliminator. Notably, the cost of generating a single SQL query is still minimal, ranging from 9% to 58% compared to other GPT-3.5 based methods due to zero-shot prompts. Although our method involves executing SQL queries, it is more efficient than other approaches. The use of a zero-shot prompt significantly reduces the overall execution time by minimizing the prompt length compared to few-shot methods.

However, it's important to acknowledge the influence of dataset size on cost efficiency. With a relatively small number of queries (only 1534) in the development set, our framework's expenses appear higher. Nonetheless, when extrapolated to larger datasets, such as facing thousands or even tens of thousands of user queries, the overhead required by our approach remains minimal.

Unlike other methods based on GPT-4 that yield noteworthy results but at high costs, SEA-SQL is more affordable. It leverages GPT-3.5 and requires only 0.9% to 5.3% of the expenses associated with GPT-4 for generating a single SQL query, yet it achieves competitive results. Consequently, SEA-SQL emerges as more aligned with practical needs, offering a cost-effective solution without compromising performance.

5.6 Robustness

We further assessed the robustness of SEA-SQL by validating it on the Spider-realistic dataset. In particular, we conducted experiments on the untrained Spider-realistic dataset, the results of which are detailed in Table 6. Remarkably, our framework showcases superior performance even when compared to the GPT-4-based approach. This underscores the robustness and generalization capabilities of our zero-shot prompts, suggesting its efficacy across diverse datasets and scenarios, thus surpassing the limitations typically associated with few-shot prompts.

5.7 LLM backbones

To further verify the effectiveness of SEA-SQL, we conducted experiments across various LLM backbones. These experiments include tests on different generation models, such as GPT-4o-mini, Qwen-2.5-72b-it (Instruct) [37], and Llama-3.1-70b-it [38], as well as various bias elimimnation models, like Gemma2-7B [39] and Llama2-7B [40]. The results, presented in Table 7, clearly demonstrate that SEA-SQL consistently achieves excellent performance across all LLM backbones. Notably, SEA-SQL, when based on Llama-3.1-70b-it and Qwen-2.5-72b-it, even perform better than the ones with GPT-3.5 and GPT-4o-mini. However, to ensure a fair comparison, we chose GPT-3.5 as our primary experimental backbone.

5.8 Error analysis

To better understand the constraints of our approach, we conducted an evaluation on the development set of the BIRD datasets. Following the methodology of DIN-SQL [17], we classified errors into five categories: schema linking errors, JOIN errors, GROUP BY errors, nesting errors, and other errors. Schema linking errors encompass the misuse of incorrect tables, columns, or values. JOIN errors are characterized by the use of incorrect columns or tables for

Table 5 The cost of different methods on the BIRD development set

Method	Model	Pre ($)	Generate / Query ($)	All ($)	GPU-Time / Query (s)	API-Time / Query (s)	Full-Time / Query (s)
GPT-4	GPT-4	–	0.0351	54	–	1.17	1.17
DIN-SQL	GPT-4	–	0.7867	1207	–	13.86	13.86
DAIL-SQL	GPT-4	–	0.1232	189	–	12.29	12.29
MAC-SQL	GPT-4	–	0.2151	330	–	15.84	15.84
DIN-SQL	GPT-3.5	–	0.0776	119	–	13.86	13.86
DAIL-SQL	GPT-3.5	–	0.0113	**17**	–	12.29	12.29
MAC-SQL	GPT-3.5	–	0.0209	32	–	15.84	15.84
SEA-SQL	GPT-3.5	44.78	**0.0065**	55	1.09	7.37	8.46

[2] We use gpt-3.5-turbo-16k-0613 to calculate the cost of GPT-3.5 and gpt-4(8k) to calculate the cost of GPT-4.

Table 6 SEA-SQL and baselines on spider-realistic

Method	Model	EX
DIN-SQL	GPT-4	78.1
DAIL-SQL	GPT-4	76.0
MAC-SQL	GPT-4	78.1
C3	GPT-3.5	75.4
DIN-SQL	GPT-3.5	65.0
DAIL-SQL	GPT-3.5	69.3
MAC-SQL	GPT-3.5	70.3
SEA-SQL(ours)	GPT-3.5 + Mistral-7B	**80.1**

Table 7 The results of different backbones on the BIRD development set

Generation model	Bias elimination model	EX				VES			
		Simple	Mod.	Chall.	All	Simple	Mod.	Chall.	All
GPT-3.5	Mistral-7B	64.11	47.74	31.94	56.13	69.63	51.71	33.72	60.83
GPT-4o-mini	Mistral-7B	65.30	44.73	36.11	56.32	68.52	46.88	36.70	58.98
Qwen-2.5-72b-it	Mistral-7B	64.86	49.46	35.42	57.43	68.22	58.20	38.04	60.53
Llama-3.1-70b-it	Mistral-7B	**66.70**	**53.76**	**47.22**	**60.95**	**70.45**	**62.30**	**51.41**	**66.19**
GPT-3.5	Gemma2-7B	62.70	48.17	33.33	55.54	68.05	51.78	33.54	59.88
GPT-3.5	Llama2-7B	62.59	43.66	31.25	53.91	67.19	47.10	32.20	57.82

JOIN operations. GROUP BY errors arise when incorrect columns are used or when GROUP BY is not utilized. Nesting errors involve the application of incorrect sub-queries or improper operator usage.

As illustrated in Fig. 6, schema linking errors are the most common, accounting for 37% of errors in the BIRD dataset. Additionally, JOIN errors also represent a substantial segment, accounting for 36% of the total errors, a percentage nearly equivalent to that of schema linking errors. Conversely, nesting and GROUP BY errors represent a minor portion of the total errors. These observations underscore two primary limitations of our approach: first, it may select incorrect tables or columns when faced with a large amount of irrelevant information; second, it struggles with handling complex SQL queries.

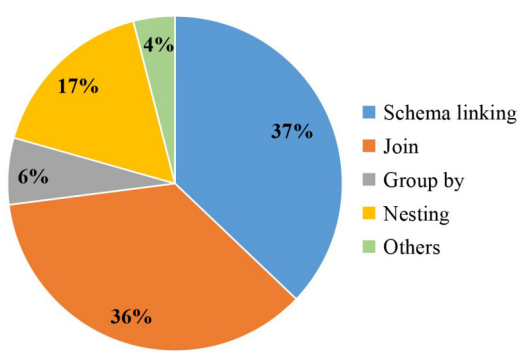


Fig. 6 Error Distributions of SEA-SQL on the BIRD development set

6 Conclusion

In this paper, we proposed SEA-SQL, a powerful framework that utilized GPT-3.5 to generate SQL queries and is competitive to GPT-4. We implemented this framework with a smaller LLM (Mistral-7B) to eliminate the biases inherent in the larger LLM (GPT-3.5). Additionally, we employed GPT-3.5 for dynamic execution adjustment, which reflects and corrects unexecutable SQL queries. Experimental results on two datasets demonstrated the effectiveness of our framework. By using a smaller LLM to eliminate bias, our approach yields better results compared to generating SQL queries directly.

Acknowledgements

This work was supported by the National Natural Science Foundation of China (Grant Nos. 62272054, 62192784, 62172056), the Beijing Nova Program (No. 20230484319), and the Xiaomi Young Talents Program.

Competing interests

The authors declare that they have no competing interests or financial conflicts to disclose.

Appendix

A Further bias elimination

After dynamic execution adjustment, new SQL queries are generated by the LLM. To demonstrate that these queries do not introduce new

Table A1 The results of the further refinement on the BIRD development set

	EX				VES			
	Simple	Mod.	Chall.	All	Simple	Mod.	Chall.	All
SEA-SQL (w/o further bias elimination)	64.11	47.74	31.94	**56.13**	69.63	51.73	33.72	**60.83**
SEA-SQL (w/ further bias elimination)	63.89	47.31	31.94	55.87	69.28	51.63	32.81	60.50

Table A2 The results from using few-shot examples on the BIRD development set. Where "Generate" refers to the cost of generating each SQL query ($), while "Full-T" represents the time taken to generate each SQL query (s)

	EX				VES				Generate / Query ($)	Full-Time / Query (s)
	Simple	Mod.	Chall.	All	Simple	Mod.	Chall.	All		
SEA-SQL (0-shot)	64.11	47.74	31.94	56.13	69.63	51.73	33.72	60.83	**0.0065**	**8.46**
SEA-SQL (1-shot)	64.11	47.96	**34.03**	56.39	70.23	52.99	**36.28**	61.82	0.0100	9.01
SEA-SQL (3-shot)	**64.76**	**48.82**	33.33	**56.98**	**71.03**	**54.15**	34.76	**62.51**	0.0130	13.98

biases, we conducted an experiment on the BIRD dataset by applying further bias elimination process after the dynamic execution adjustment. The results, shown in Table A1, indicate a decline in performance after this process. This decline is primarily because the dynamic execution adjustment maintains the original style of the SQL, ensuring it executable without introducing biases or syntax errors. Further bias elimination did not improve the results and might make the SQL non-executable.

B Few-shot prompts

Previous work suggests that few-shot prompts may be more effective in leveraging LLMs for data-related tasks [41]. Consequently, we explored whether using few-shot prompts would yield greater improvements, as shown in Table A2. The results indicate that using a 1-shot approach achieves better outcomes compared to zero-shot, and a 3-shot approach yields even higher results. However, the computational cost significantly increases with more examples. Given these costs, we continue to use zero-shot prompts for our main experiments. However, in scenarios where accuracy is a priority, using few-shot prompts tends to be more effective.

C Prompts

Here we present out prompts utilized for each module, including SQL generation, adaptive bias elimination and dynamic execution adjustment.

C.1 SQL generation

```
### Complete sqlite SQL query only and with no explanation.
### Sqlite SQL tables, with their properties:
#
{schema}
#
### Question: {question}
SELECT
```

C.2 Adaptive bias elimination

```
Below is an instruction that describes a task, paired with an input that provides further context. Write a response that appropriately completes the request.

### Instruction: This represents the SQLite SQL query that has been generated in response to the given question, along with the resulting outcome after executing the query.
Please judge its correctness based on the execution result and the explanation for the question. If it's incorrect, output the correct sqlite SQL query; otherwise, output the original sqlite SQL query.

### Input:
### Sqlite SQL tables, with their properties:
#
{schema}
#
### Question: {question}
### SQLite SQL query: {SQL}
### Run results: {result}

### Response:
```

C.3 Dynamic execution adjustment

```
### SQL: (error bias eliminated SQL)
### Error message: (error message of the bias eliminated SQL)
### Error reason: (error reason of the bias eliminated SQL)
### New SQL: (the corrected new SQL_1)
...
```

```
### Error message: (error message of SQL_i)
### Error reason: (error reason of the SQL_i)
### New SQL: (the corrected new SQL_{i+1})
### Error message: (error message of SQL_{i+1})
```

Here "previous_information" has the following format:

C.3.1 Reflect

```
### Here is a sqlite SQL query that resulted from a question, but it produced an error when executed. What do you think is the possible reason for this SQL error?

### Sqlite SQL tables, with their properties:
#
{schema}
#
### Question: {question}
{previous_information}
### Error Reason:
```

C.3.2 Correct

```
### Here is a sqlite SQL query that resulted from a question, but it produced an error when executed. Please correct it with no explanation.

### Sqlite SQL tables, with their properties:
#
{schema}
#
### Question: {question}
{previous_information}
### Error Reason: {reason}
### Correct SQL: SELECT
```

References

[1] Cai R, Xu B, Zhang Z, Yang X, Li Z, Liang Z. An encoder-decoder framework translating natural language to database queries. In: Proceedings of the 27th International Joint Conference on Artificial Intelligence. 2018, 3977–3983

[2] Rai D, Wang B, Zhou Y, Yao Z. Improving generalization in language model-based text-to-SQL semantic parsing: two simple semantic boundary-based techniques. In: Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers). 2023, 150–160

[3] Li H, Zhang J, Li C, Chen H. RESDSQL: decoupling schema linking and skeleton parsing for text-to-SQL. In: Proceedings of the 37th AAAI Conference on Artificial Intelligence. 2023, 13067–13075

[4] Zeng L, Parthasarathi S H K, Hakkani-Tur D. N-best hypotheses reranking for text-to-SQL systems. In: Proceedings of 2022 IEEE Spoken Language Technology Workshop (SLT). 2023, 663–670

[5] Liu Q, Yang D, Zhang J, Guo J, Zhou B, Lou J G. Awakening latent grounding from pretrained language models for semantic parsing. In: Proceedings of Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021. 2021, 1174–1189

[6] Hui B, Shi X, Geng R, Li B, Li Y, Sun J, Zhu X. Improving text-to-SQL with schema dependency learning. 2021, arXiv preprint arXiv: 2103.04399

[7] Yu T, Yasunaga M, Yang K, Zhang R, Wang D, Li Z, Radev D. SyntaxSQLNet: syntax tree networks for complex and cross-DomainText-to-SQL task. 2018, arXiv preprint arXiv: 1810.05237

[8] Wang B, Shin R, Liu X, Polozov O, Richardson M. RAT-SQL: relation-aware schema encoding and linking for text-to-SQL parsers. In: Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics. 2020, 7567–7578

[9] Gan Y, Chen X, Xie J, Purver M, Woodward J R, Drake J, Zhang Q. Natural SQL: making SQL easier to infer from natural language specifications. In: Proceedings of Findings of the Association for Computational Linguistics: EMNLP 2021. 2021, 2030–2042

[10] Yin P, Neubig G. A syntactic neural model for general-purpose code generation. In: Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). 2017, 440–450

[11] Scholak T, Schucher N, Bahdanau D. PICARD: parsing incrementally for constrained auto-regressive decoding from language models. In: Proceedings of 2021 Conference on Empirical Methods in Natural Language Processing. 2021, 9895–9901

[12] Suhr A, Chang M W, Shaw P, Lee K. Exploring unexplored generalization challenges for cross-database semantic parsing. In: Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics. 2020, 8372–8388

[13] Li J, Hui B, Qu G, Yang J, Li B, Li B, Wang B, Qin B, Geng R, Huo N, Zhou X, Ma C, Li G, Chang K C C, Huang F, Cheng R, Li Y. Can LLM already serve as A database interface? A big bench for large-scale database grounded text-to-SQLs. In: Proceedings of the 37th International Conference on Neural Information Processing Systems. 2023, 1835

[14] Liu A, Hu X, Wen L, Yu P S. A comprehensive evaluation of ChatGPT's zero-shot text-to-SQL capability. 2023, arXiv preprint arXiv: 2303.13547

[15] Chang S, Fosler-Lussier E. How to prompt LLMS for text-to-SQL: a study in zero-shot, single-domain, and cross-domain settings. 2023, arXiv preprint arXiv: 2305.11853

[16] Dong X, Zhang C, Ge Y, Mao Y, Gao Y, Chen L, Lin J, Lou D. C3: zero-shot text-to-SQL with ChatGPT. 2023, arXiv preprint arXiv: 2307.07306

[17] Pourreza M, Rafiei D. DIN-SQL: decomposed in-context learning of text-to-SQL with self-correction. In: Proceedings of the 37th International Conference on Neural Information Processing Systems. 2023, 1577

[18] Gao D, Wang H, Li Y, Sun X, Qian Y, Ding B, Zhou J. Text-to-SQL empowered by large language models: a benchmark evaluation. Proceedings of the VLDB Endowment, 2024, 17(5): 1132−1145

[19] Zhou X, Sun Z, Li G. DB-GPT: large language model meets database. Data Science and Engineering, 2024, 9(1): 102−111

[20] Yao S, Zhao J, Yu D, Du N, Shafran I, Narasimhan K R, Cao Y.

ReAct: synergizing reasoning and acting in language models. In: Proceedings of the 11th International Conference on Learning Representations. 2023, 1–33

[21] Wang B, Ren C, Yang J, Liang X, Bai J, Zhang Q W, Yan Z, Li Z. MAC-SQL: a multi-agent collaborative framework for text-to-SQL. 2023, arXiv preprint arXiv: 2312.11242

[22] Wang X, Wei J, Schuurmans D, Le Q V, Chi E H, Narang S, Chowdhery A, Zhou D. Self-consistency improves chain of thought reasoning in language models. In: Proceedings of the 11th International Conference on Learning Representations. 2023, 1–24

[23] Lin X V, Socher R, Xiong C. Bridging textual and tabular data for cross-domain text-to-SQL semantic parsing. In: Proceedings of Findings of the Association for Computational Linguistics: EMNLP 2020. 2020, 4870–4888

[24] Wan X, Han X. Efficient top-*k* frequent itemset mining on massive data. Data Science and Engineering, 2024, 9(2): 177−203

[25] Li H, Zhang J, Liu H, Fan J, Zhang X, Zhu J, Wei R, Pan H, Li C, Chen H. CodeS: towards building open-source language models for text-to-SQL. Proceedings of the ACM on Management of Data, 2024, 2(3): 127

[26] Wang J, Zhao W, Tu X, He T. A novel dense retrieval framework for long document retrieval. Frontiers of Computer Science, 2023, 17(4): 174609

[27] Wei J, Wang X, Schuurmans D, Bosma M, Ichter B, Xia F, Chi E H, Le Q V, Zhou D. Chain-of-thought prompting elicits reasoning in large language models. In: Proceedings of the 36th International Conference on Neural Information Processing Systems. 2024, 1800

[28] Yu T, Zhang R, Yang K, Yasunaga M, Wang D, Li Z, Ma J, Li I, Yao Q, Roman S, Zhang Z, Radev D R. Spider: a large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task. In: Proceedings of 2018 Conference on Empirical Methods in Natural Language Processing. 2018, 3911–3921

[29] Deng X, Awadallah A H, Meek C, Polozov O, Sun H, Richardson M. Structure-grounded pretraining for text-to-SQL. In: Proceedings of 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies. 2021, 1337–1350

[30] Zhong R, Yu T, Klein D. Semantic evaluation for text-to-SQL with distilled test suites. In: Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP). 2020, 396–411

[31] Xu Y, Xie L, Gu X, Chen X, Chang H, Zhang H, Chen Z, Zhang X, Tian Q. QA-LoRA: quantization-aware low-rank adaptation of large language models. In: Proceedings of the 12th International Conference on Learning Representations, 2024

[32] Aminabadi R Y, Rajbhandari S, Awan A A, Li C, Li D, Zheng E, Ruwase O, Smith S, Zhang M, Rasley J, He Y. DeepSpeed- inference: enabling efficient inference of transformer models at unprecedented scale. In: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis. 2022, 1–15

[33] Dao T, Fu D Y, Ermon S, Rudra A, Ré C. FLASHATTENTION: fast and memory-efficient exact attention with IO-awareness. In: Proceedings of the 36th International Conference on Neural Information Processing Systems. 2022, 1189

[34] Jiang W, Ning B, Li G, Bai M, Jia X, Wei F. Graph-decomposed *k*-NN searching algorithm on road network. Frontiers of Computer Science, 2024, 18(3): 183609

[35] Jin P, Chu Z, Liu G, Luo Y, Wan S. Optimizing B^+-tree for hybrid memory with in-node hotspot cache and eADR awareness. Frontiers of Computer Science, 2024, 18(5): 185606

[36] Li F, Zhang T, Cui S, Liu H, Ren Z, Di D, Wang X, Zhang P, Yu G. A sampling method based on forecasting and combinatorial optimization for high performance A/B testing. Frontiers of Computer Science, 2023, 17(6): 176616

[37] Yang A, Yang B, Hui B, Zheng B, Yu B, Zhou C, Li C, Li C, Liu D, Huang F, Dong G, Wei H, Lin H, Tang J, Wang J, Yang J, Tu J, Zhang J, Ma J, Yang J, Xu J, Zhou J, Bai J, He J, Lin J, Dang K, Lu K, Chen K, Yang K, Li M, Xue M, Ni N, Zhang P, Wang P, Peng R, Men R, Gao R, Lin R, Wang S, Bai S, Tan S, Zhu T, Li T, Liu T, Ge W, Deng X, Zhou X, Ren X, Zhang X, Wei X, Ren X, Liu X, Fan Y, Yao Y, Zhang Y, Wan Y, Chu Y, Liu Y, Cui Z, Zhang Z, Guo Z, Fan Z. Qwen2 technical report. 2024, arXiv preprint arXiv: 2407. 1067, 1: 2024

[38] Dubey A, Jauhri A, Pandey A, Kadian A, Al-Dahle A, et al. The Llama 3 herd of models. 2024, arXiv preprint arXiv: 2407.21783

[39] Rivière M, Pathak S, Sessa P G, Hardin C, Bhupatiraju S, et al. Gemma 2: improving open language models at a practical size. 2024, arXiv preprint arXiv: 2408.00118

[40] Touvron H, Martin L, Stone K, Albert P, Almahairi A, Babaei Y, Bashlykov N, Batra S, Bhargava P, Bhosale S, Bikel D, Blecher L, Canton Ferrer C, Chen M, Cucurull G, Esiobu D, Fernandes J, Fu J, Fu W, Fuller B, Gao C, Goswami V, Goyal N, Hartshorn A, Hosseini S, Hou R, Inan H, Kardas M, Kerkez V, Khabsa M, Kloumann I, Korenev A, Koura P S, Lachaux M A, Lavril T, Lee J, Liskovich D, Lu Y, Mao Y, Martinet X, Mihaylov T, Mishra P, Molybog I, Nie Y, Poulton A, Reizenstein J, Rungta R, Saladi K, Schelten A, Silva R, Smith E M, Subramanian R, Tan X E, Tang B, Taylor R, Williams A, Kuan J X, Xu P, Yan Z, Zarov I, Zhang Y, Fan A, Kambadur M, Narang S, Rodriguez A, Stojnic R, Edunov S, Scialom T. Llama 2: open foundation and fine-tuned chat models. 2023, arXiv preprint arXiv: 2307.09288

[41] Zhang H, Dong Y, Xiao C, Oyamada M. Large language models as data preprocessors. In: Proceedings of Workshops at the 50th International Conference on Very Large Data Bases. 2024, 1–4



Chaofan LI is a master's student at Beijing University of Posts and Telecommunications (BUPT), China, focusing on LLM-based information retrieval and Text-to-SQL research. His work has been published in top-tier conferences including ACL and SIGIR.



Yingxia SHAO is a professor at the School of Computer Science, Beijing University of Posts and Telecommunications (BUPT), China. His research focuses on intelligent data management, large graph computing, AI4DB, and large-scale machine learning. He has published over 80 papers in top conferences and journals, with more than 40 in CCF A-class venues. He has received the ACM SIGMOD China Rising Star Award, Outstanding Dissertation Award, and the VLDB 2022 Best Paper Award.



Yawen LI is an associate professor at Beijing University of Posts and Telecommunications (BUPT), China. Her research focuses on big data, machine learning, and intelligent decision-making, with emphasis on enterprise innovation and knowledge graph evolution. She has published 77 papers, including 27 in top venues like TKDE, VLDB, and AAAI, and holds 7 patents. Her achievements have been recognized with awards including the Wu Wenjun AI Science and Technology Youth Award and the 2022 ACM Beijing Chapter Rising Star Award, and she serves on several academic committees and editorial boards.



Zheng LIU is a researcher at the Beijing Academy of Artificial Intelligence (BAAI), China. He previously served as a Lead Researcher at Microsoft Research Asia (MSRA) and Technical Expert at Huawei 2012 Labs. He has led the development of open-source models like BAAI General Embedding, and participated in the development of commercial information retrieval systems such as BING Ads, Microsoft News, and Huawei Petal Search. He has published numerous papers in top journals in the fields of natural language processing, information retrieval, and data mining.